

Πειράματα Πρώτης Εργασίας – LSH

1. Εισαγωγή

Στόχος της εργασίας ήταν η πειραματική αξιολόγηση του αλγορίθμου **LSH (Locality Sensitive Hashing)** σε datasets τύπου SIFT, και Mnist.

Η μέθοδος LSH είναι σχεδιασμένη για **ταχύτερη εύρεση κοντινών γειτονικών σημείων** σε μεγάλους χώρους υψηλής διάστασης.

2. Περιγραφή Αλγορίθμου

Χρησιμοποιήθηκε η υλοποίηση **E2LSH**, με hash functions τύπου ευκλίδειας απόστασης :

Όπου:

$$h(p) = \lfloor p \cdot v + t/w \rfloor$$

$$g(p) = ID = (r_1 h_1(p) + r_2 h_2(p) + \dots + r_k h_k(p)) \bmod M$$

- (p) : διανύσματα του dataset
- (w) : παράμετρος εύρους παραθύρου (window size)
- (t) : τυχαία μετατόπιση
- (r_i) : τυχαίοι ακέραιοι συντελεστές
- (M) : μέγεθος hash table

Το **hash table** έχει μήκος (n) και χρησιμοποιούμε το $(g(p) \bmod n)$ για την τοποθέτηση των σημείων.

3. Dataset SIFT (Non-Normalized)

Το dataset `sift_base.fvecs` περιέχει **1.000.000 διανύσματα**, με float τιμές.

Οι αποστάσεις μεταξύ διανυσμάτων είναι **της τάξης μερικών εκατοντάδων**.

3.1 Παρατηρήσεις

- Μικρό (w) → μικρός αριθμός collisions → χαμηλό **recall** και **average factor (AF)**
 - Μεγάλο (w) → υψηλό recall και AF, αλλά το πλήθος των σημείων που εξετάζονται μεγαλώνει πολύ → πολύ χαμηλό QPS → η brute-force λύση είναι καλύτερη
 - Παρατηρήθηκε ότι $(w = 4.0)$ δεν δίνει καλά αποτελέσματα λόγω μικρού αριθμού collisions
-

4. Dataset SIFT (Normalized)

Η κανονικοποίηση του dataset (normalization) επηρεάζει τη συμπεριφορά της LSH:

- Η κανονικοποίηση μειώνει την απόσταση μεταξύ διανυσμάτων
- Βελτιώνει την ακρίβεια για μικρότερα (w)
- Επιτρέπει μεγαλύτερη σταθερότητα στα QPS και Recall

Recall vs w :

5. Mnist Dataset

Αντιστοιχ αποτελέσματα παρατηρούμε και στο mnist dataset .Πολυ μικρα w μειωνουν το recall αλλα αυξανουν τα qps . Αντιθετα μεγαλα w αυξανουν το recall αλλα μειωνουν τα qps

6. Συμπεράσματα

1. Υπάρχει **trade-off** μεταξύ **recall/AF** και **QPS**:
 - Μικρό (w) → γρήγορη αναζήτηση, χαμηλό recall
 - Μεγάλο (w) → υψηλό recall, χαμηλό QPS
2. Ο αλγόριθμος LSH είναι αποτελεσματικός για **μέτριες τιμές (w)** και **μέγεθος hash table M**
3. Πέρα από κάποιο όριο (π.χ., πολύ μεγάλο (w)) η brute-force αναζήτηση μπορεί να είναι πιο αποδοτική
4. Δεν παρατηρήθηκε μεγάλο trade-off μεταξύ L και qps - recall.Εαν και λιγα L μπορει να μειωσουν το recall πολλα σιγουρα δεν βελτιωνουν την εικονα.

LSH Hypercube

1. Εισαγωγή

Στόχος της εργασίας ήταν η πειραματική αξιολόγηση του αλγορίθμου **LSH Hypercube** σε datasets τύπου SIFT, και MNIST.

Η μέθοδος Hypercube είναι σχεδιασμένη για **ταχύτερη εύρεση κοντινών γειτονικών σημείων** σε μεγάλους χώρους υψηλής διάστασης, χρησιμοποιώντας **binary hashing** και αναζήτηση στους γείτονες ενός hypercube.

2. Περιγραφή Αλγορίθμου

Χρησιμοποιήθηκε η υλοποίηση **Hypercube LSH**, με hash functions τύπου ευκλείδειας απόστασης:

$$h(p) = \lfloor v p + t / W \rfloor > \text{median}(h(z) \text{ για κάθε } z \text{ σημείο του dataset}) ? 1 : 0$$

Όπου:

- p : διανύσματα του dataset
- v_i : τυχαίο διάνυσμα $N(0,1)$

- t : τυχαία μετατόπιση
- w : παραθυρο
- k : αριθμός διαστάσεων του hypercube

Το αποτέλεσμα $h(p_-)$ είναι ένα **0 η 1** , που καθορίζει τη θέση του σημείου στο **vertex του hypercube**.

Η αναζήτηση γίνεται εξετάζοντας τον αρχικό vertex και τους **γείτονες σε Hamming απόσταση αρχικά 1 το πολύ μέχρι k εξετάζοντας το πολύ probes**.

3. Dataset SIFT (Non-Normalized)

Το dataset `sift_base.fvecs` περιέχει **1.000.000 διανύσματα**, με float τιμές.

Οι αποστάσεις μεταξύ διανυσμάτων είναι **της τάξης μερικών εκατοντάδων**.

3.1 Παρατηρήσεις

- Μικρό k_{proj} → μικρός αριθμός bits → πολλά collisions → μεγάλο AF και Recall
 - Μεγάλο k_{proj} → μεγάλος αριθμός bits → λιγότερα collisions → χαμηλότερο AF και Recall.
 - Ο αριθμός των **probes** επηρεάζει άμεσα το recall και τον χρόνο: περισσότερα probes → καλύτερο recall αλλά χαμηλότερο QPS
-

4. Dataset SIFT (Normalized)

Η κανονικοποίηση του dataset επηρεάζει την απόδοση του Hypercube LSH:

- Η κανονικοποίηση μειώνει τις αποστάσεις μεταξύ διανυσμάτων
 - Επιτρέπει υψηλότερο recall με μικρότερο αριθμό probes
 - Σταθεροποιεί το QPS και την απόδοση στις διαφορετικές τιμές k_{proj} .
 - Μπορεί να έχει χειρότερο qps.
-

5. Mnist Dataset

Παρόμοια συμπεριφορά παρατηρείται και στο MNIST dataset:

- Πολύ μικρό k → μειωμένο recall, αλλά υψηλό QPS
 - Μεγάλο k → αυξημένο recall, αλλά μειωμένο QPS
 - Αύξηση του αριθμού **probes** αυξάνει το recall, αλλά μειώνει το QPS
-

6. Συμπεράσματα

1. Υπάρχει **trade-off** μεταξύ **recall/AF** και **QPS**:
 - Μικρό k και πολλοί probes $\rightarrow$ αργή αναζήτηση, υψηλότερο recall
 - Μεγάλο k ή λίγοι probes $\rightarrow$ χαμηλότερο recall, χαμηλό QPS
2. Ο αλγόριθμος Hypercube είναι αποτελεσματικός για **μέτριες τιμές k** και **probes**
3. Πέρα από κάποιο όριο (πολύ μικρό k ή πολλά probes) η brute-force αναζήτηση μπορεί να είναι πιο αποδοτική
4. Ο συνδυασμός **k και probes** καθορίζει την ισορροπία μεταξύ ταχύτητας και ακρίβειας